

Homework 6

Due at 11:59 pm, June 1st, 2026

Please carefully explain and justify your answers. Plots should be clearly labeled, with axes, legends, and units when appropriate. Include your MATLAB code or Python code as part of your submission. You may use MATLAB's Optimization Toolbox, MPC Toolbox, CasADi, or another appropriate optimization tool, but you should clearly explain how your optimization problems are formulated.

1 Tracking MPC with Hard and Soft Constraints

Consider the mass-spring system

$$\begin{aligned}\dot{x}_1 &= x_2, \\ \dot{x}_2 &= -\frac{k}{m}x_1 + \frac{1}{m}u,\end{aligned}$$

where x_1 is the position of the mass, x_2 is the velocity, and u is the control force. Let

$$m = 1, \quad k = 1,$$

and use the sampling time

$$T_s = 0.1 \text{ sec.}$$

- (a) Derive the continuous-time state-space model

$$\dot{x} = A_c x + B_c u,$$

where

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Then obtain the discrete-time model

$$x_{k+1} = A_d x_k + B_d u_k.$$

You may use MATLAB's `c2d` command to compute A_d and B_d .

- (b) Let r_k denote a position command, and define the tracking error

$$e_k = x_{1,k} - r_k.$$

We will use an incremental-input formulation. Define

$$\Delta x_k = x_{k+1} - x_k, \quad \Delta u_k = u_{k+1} - u_k,$$

and introduce the augmented state

$$z_k = \begin{bmatrix} \Delta x_{1,k} \\ \Delta x_{2,k} \\ e_k \\ u_k \\ x_{1,k} \end{bmatrix}.$$

Derive the augmented model

$$z_{k+1} = Az_k + B\Delta u_k.$$

You may use the following MATLAB code as a starting point, but you should explain where the matrices come from:

```
A = [Ad, zeros(2,3);
      1, 0, 1, 0, 0;
      0, 0, 0, 1, 0;
      1, 0, 0, 0, 1];

B = [Bd;
      0;
      1;
      0];
```

(c) Consider the finite-horizon tracking MPC problem with horizon

$$N = 15.$$

Use the cost function

$$J = z_N^\top P z_N + \sum_{k=0}^{N-1} \left(z_k^\top Q z_k + \Delta u_k^\top R \Delta u_k \right),$$

where

$$Q = \text{diag}(0, 0, 1, 0, 0), \quad R = 1.$$

The matrix P should be chosen using the discrete-time LQR solution for the subsystem involving $(\Delta x_1, \Delta x_2, e)$:

```
Q = diag([0, 0, 1, 0, 0]);
R = 1;

[K, Pdx_e, E] = dlqr(A(1:3,1:3), B(1:3), diag([0,0,1]), 1);
P = blkdiag(Pdx_e, zeros(2,2));
```

Write the MPC optimization problem in the quadratic programming form

$$\min_U \frac{1}{2} U^\top H U + q^\top U$$

subject to

$$GU \leq W + Tz_0,$$

where

$$U = \begin{bmatrix} \Delta u_0 \\ \Delta u_1 \\ \vdots \\ \Delta u_{N-1} \end{bmatrix}.$$

The first optimal control increment is selected by

$$\Delta u_0 = I_{\text{MPC}} U.$$

The actual control input should then be updated according to

$$u_{k+1} = u_k + \Delta u_k.$$

(d) Impose the hard constraints

$$|x_1| \leq 0.2, \quad |u| \leq 0.2.$$

The decision variable in the MPC problem is Δu , while the actual control input u is one of the augmented states. You may use the following constraint setup:

```
lN = 10; % a large number used for unconstrained variables

xlim.max = [lN, lN, lN, 0.2, 0.2]; % [dx1, dx2, e, u, x1]
xlim.min = -xlim.max;

ulim.max = lN;
ulim.min = -ulim.max; % the optimization input is Delta u
```

Develop a function with the following template:

```
function [H, L, G, W, T, IMPC] = formQPMatrices(A, B, Q, R, P, xlim, ulim, N)

% Form the prediction matrices, cost matrices, and constraint matrices
% for the finite-horizon MPC problem.
%
% The QP should have the form
%
%      minimize    1/2 U' H U + q' U
%      subject to   G U <= W + T z0
%
% where q = L z0.
%
% IMPC should select the first control increment:
%
%      Delta_u0 = IMPC * U
end
```

(e) Simulate the closed-loop response of the MPC controller. Use MATLAB's `quadprog` to solve the QP at each sampling time. Between control updates, you may simulate the continuous-time plant using `ode45`.

Use a sequence of position commands between -0.19 and 0.19 , and include one command to

$$r = -0.25,$$

which lies outside the feasible position range.

Plot:

- $x_1(t)$ and $r(t)$ on the same plot;
- the position constraint bounds $x_1 = \pm 0.2$;
- the control input $u(t)$.

Discuss what happens when the reference command is infeasible.

(f) Test robustness by simulating the plant with

$$m_{\text{plant}} = 0.8, \quad k_{\text{plant}} = 1.2,$$

while keeping the nominal model

$$m = 1, \quad k = 1$$

inside the MPC controller.

First repeat the closed-loop simulation with the original input constraint

$$|u| \leq 0.2.$$

Then increase the control authority to

$$|u| \leq 0.25$$

and repeat the simulation again.

Compare the results. Discuss how plant-model mismatch affects tracking performance and constraint satisfaction, and whether the additional control authority improves the closed-loop response.

The continuous-time mass-spring dynamics may be implemented using the following function:

```
function xdot = msd(t, x, u, m_msd, k_msd)

x1 = x(1);
x2 = x(2);

xdot = zeros(2,1);
xdot(1) = x2;
xdot(2) = -(k_msd/m_msd)*x1 + (1/m_msd)*u;

end
```

2 (Optional) Nonlinear MPC for Spacecraft Attitude Control (extra 50 points)

The attitude dynamics of a spacecraft are represented by

$$J_1 \dot{\omega}_1 + (J_3 - J_2) \omega_2 \omega_3 = u_1,$$

$$J_2 \dot{\omega}_2 + (J_1 - J_3) \omega_1 \omega_3 = u_2,$$

$$J_3 \dot{\omega}_3 + (J_2 - J_1) \omega_1 \omega_2 = u_3.$$

The Euler-angle kinematics are

$$\frac{d}{dt} \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix} = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi \sec \theta & \cos \phi \sec \theta \end{bmatrix} \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix}.$$

Let

$$x = [\omega_1 \quad \omega_2 \quad \omega_3 \quad \phi \quad \theta \quad \psi]^\top, \quad u = [u_1 \quad u_2 \quad u_3]^\top.$$

Use

$$J_1 = 918, \quad J_2 = 920, \quad J_3 = 1365.$$

This problem is inspired by a real-time iteration/SQP formulation for NMPC. To keep the assignment focused on formulation, implementation, and interpretation, you may use **fmincon**, CasADi, or another nonlinear optimization solver. You are not required to implement all gradients, Hessians, and KKT residuals from scratch unless you choose to do so.

- (a) Implement the continuous-time nonlinear dynamics

$$\dot{x} = f_c(x, u).$$

Then discretize the dynamics using explicit Euler integration:

$$x_{k+1} = x_k + T_s f_c(x_k, u_k),$$

where

$$T_s = 2 \text{ sec.}$$

You may use the following code as a starting point:

```
function xdot = attitude_dynamics(x, u, J)

% x = [w1; w2; w3; phi; theta; psi]
% u = [u1; u2; u3]
% J = diag([J1, J2, J3])

w = x(1:3);
phi = x(4);
theta = x(5);

wdot = J \ (-mcross(w)*J*w + u);

S = zeros(3,3);
S(1,1) = 1;
S(1,2) = sin(phi)*tan(theta);
S(1,3) = cos(phi)*tan(theta);

S(2,1) = 0;
S(2,2) = cos(phi);
S(2,3) = -sin(phi);
```

```

S(3,1) = 0;
S(3,2) = sin(phi)/cos(theta);
S(3,3) = cos(phi)/cos(theta);

eulerdot = S*w;

xdot = [wdot; eulerdot];

end

```

```

function ux = mcross(u)

ux = [ 0,    -u(3),  u(2);
       u(3),  0,    -u(1);
      -u(2), u(1),   0   ];

end

```

```

function xnext = attitude_discrete(x, u, J, Ts)

xnext = x + Ts*attitude_dynamics(x, u, J);

end

```

(b) Formulate a nonlinear MPC problem with prediction horizon

$$N = 10.$$

Use the cost function

$$J = \frac{1}{2}(x_N - r)^\top P(x_N - r) + \frac{1}{2} \sum_{k=0}^{N-1} \left[(x_k - r)^\top Q(x_k - r) + u_k^\top R u_k \right],$$

where

$$Q = \text{diag}(100, 100, 100, 10, 10, 10),$$

$$R = \text{diag}(0.1, 0.1, 0.1).$$

Here r is the desired attitude state.

Choose P as the discrete-time LQR terminal penalty obtained by linearizing the discrete-time model at

$$x = 0, \quad u = 0.$$

Clearly write the resulting NMPC problem in the form

$$\min_{x_1, \dots, x_N, u_0, \dots, u_{N-1}} J$$

subject to

$$x_{k+1} = f(x_k, u_k), \quad k = 0, \dots, N-1.$$

(c) Add the input constraints

$$-1 \leq u_i \leq 1, \quad i = 1, 2, 3.$$

Also add state constraints of the form

$$x_{\min} \leq x_k \leq x_{\max}.$$

Clearly state how these constraints are included in the NMPC problem.

(d) Simulate the NMPC controller for

200 sec

using the initial condition

$$x(0) = [0 \ 0 \ 0 \ 15^\circ \ 30^\circ \ 20^\circ]^\top$$

and the target

$$r = 0.$$

Plot:

- angular velocities $\omega_1, \omega_2, \omega_3$;
- Euler angles ϕ, θ, ψ ;
- control torques u_1, u_2, u_3 .

Include the input constraint bounds in the control plot.

(e) Repeat the simulation using the following two state-constraint settings. Recall that

$$x = [\omega_1 \ \omega_2 \ \omega_3 \ \phi \ \theta \ \psi]^\top.$$

$$\text{Case 1:} \quad x_{\max} = 10 \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad x_{\min} = -10 \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0.01 \\ 0.01 \\ 0.01 \end{bmatrix}.$$

$$\text{Case 2:} \quad x_{\max} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad x_{\min} = - \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

In both cases, use the input constraints

$$-1 \leq u_i \leq 1, \quad i = 1, 2, 3.$$

Plot the following quantities for both cases:

- angular velocities $\omega_1, \omega_2, \omega_3$;
- Euler angles ϕ, θ, ψ ;
- control torques u_1, u_2, u_3 , together with the upper and lower input bounds;
- solver residual, optimality measure, or constraint violation measure, if available.

Compare the closed-loop responses. Discuss how the state constraints affect maneuver speed, control saturation, convergence behavior, and solver residual.

You may implement the NMPC problem using MATLAB's `fmincon`, CasADi, or another non-linear optimization solver. If you use `fmincon`, the following function templates may be helpful.

```
function [u0, Xopt, Uopt, exitflag] = solve_nmpc_attitude(x0, xref, Xguess, Uguess,
    params)

% params.N
% params.Ts
% params.J
% params.Q
% params.R
% params.P
% params.xlb, params.xub
% params.ulb, params.uub

N = params.N;
nx = 6;
nu = 3;

z0 = [Xguess(:); Uguess(:)];

obj = @(z) nmpc_cost(z, x0, xref, params);
nonlcon = @(z) nmpc_constraints(z, x0, params);

opts = optimoptions('fmincon', ...
    'Display','none', ...
    'Algorithm','sqp', ...
    'MaxIterations',100);

[zopt,~,exitflag] = fmincon(obj,z0,[],[],[],[],[],[],nonlcon,opts);

Xopt = reshape(zopt(1:nx*N), nx, N);
Uopt = reshape(zopt(nx*N+1:end), nu, N);

u0 = Uopt(:,1);

end
```

```
function Jcost = nmpc_cost(z, x0, xref, params)

% Complete this function.
%
% The function should compute
%
% 1/2 (x_N - r)' P (x_N - r)
```



```

%   + 1/2 sum_{k=0}^{N-1} [(x_k-r)' Q (x_k-r) + u_k' R u_k].
%
% Note that x0 is fixed and is not part of the optimization vector.
end

```

```

function [c, ceq] = nmpc_constraints(z, x0, params)

% Complete this function.
%
% ceq should contain the equality constraints from the system dynamics.
% c should contain the inequality constraints for states and inputs.
%
% The constraints should be written in the standard fmincon form:
%
%       c(z) <= 0,
%       ceq(z) = 0.
end

```

““